GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M7: ESP32 Software and Communication

Company: Board&Byte **Group:** Group-8 **Date:** 6 November 2026

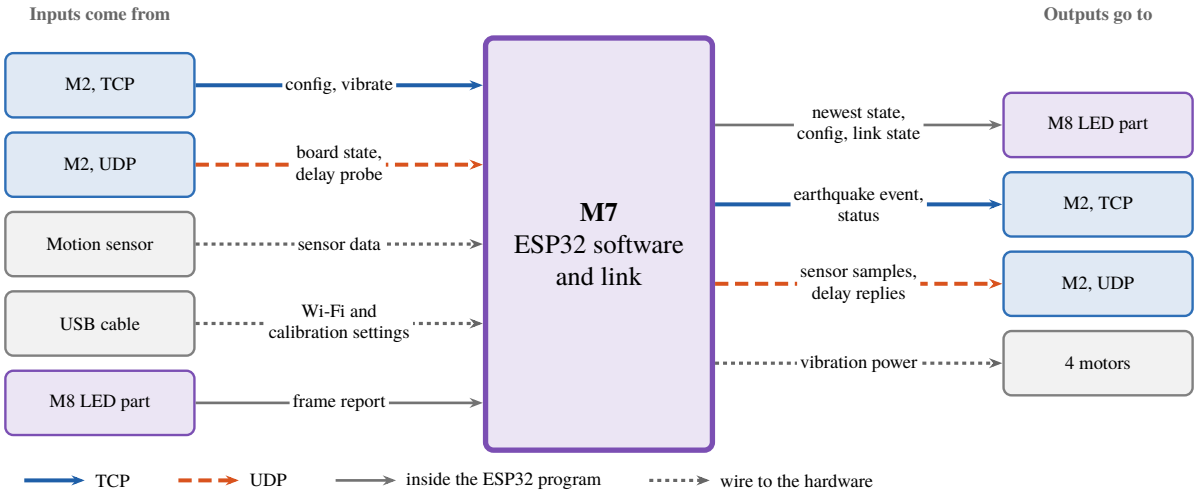
Name	Student No.	Name	Student No.
Tuğberk Akbulut	230104004070	Hasan Alperen Elmas	1801042604
Hasan Hüseyin Özdemir	230104004048	Muhib Taha Boy	230104004165

1. Purpose

M7 is the part of the ESP32 program that talks to the outside world. It gets the game state from the main computer over Wi-Fi and gives it to the LED part (M8). It reads the motion sensor and finds the strength and the place of a hit on the board. It drives the vibration motors. It also reports the health of the board and the sensor signal to the admin.

Runs on	Technology	Requirements
ESP32 in the LED board, in one Arduino program together with M8	C++ with Arduino for ESP32, TCP and UDP over Wi-Fi with MessagePack messages, a motion sensor with accelerometer and gyroscope, 4 vibration motors	R7 R8 R10 R11

M7 has links on three sides: the server over Wi-Fi, the M8 part in the same program, and the hardware under the board. The figure shows each of them.



2. Requirements covered

M7 takes part in four requirements. The last column says what will work when M7 is finished.

ID	Requirement	What M7 does for it	When M7 is done
R7	Earthquake	Finds hand hits and motor shaking with the sensor, with a strength from 0 to 10 and a zone in a 3 × 3 grid. Drives the motors for the admin button	A hit on the table starts an earthquake in the game
R8	Physical LED board	Brings the game state to the board over Wi-Fi. One ESP32 for the LEDs, the sensor and the motors	The board shows the live game without a cable to the computer
R10	Network and measurement	Delay replies, sensor samples for the seismic graph, comes back after a lost link	The admin sees the board delay and the sensor signal live
R11	Engineering requirements	PC emulators for both sides of the link	The server and the board can be tested without each other

3. Inputs and outputs

The table shows every input and output with its link. Messages that must arrive use TCP, fast and repeated data uses UDP.

Direction	Data	From / to	Content
In	Board state	M2, UDP	10 to 30 per second: phase, players (team, colour, position, flags), delivery points, obstacles, earthquake
In	Config	M2, TCP	Player colours, brightness from 0.1 to 1, power limit
In	Vibrate	M2, TCP	Motors to use, strength from 0 to 1, duration
In	Delay probe	M2, UDP	A small message with the server time, answered at once
In	Sensor	Hardware	Motion data from the accelerometer and the gyroscope
In	Settings	USB cable	Wi-Fi name and password, backup server address, calibration values
In	Frame report	M8	Frame rate and the power estimate of the last frames
Out	Newest state	M8	Latest board state, config and link state, in the same program
Out	Earthquake event	M2, TCP, to M1 and M6	Source (hand or motors), strength from 0 to 10, centre, zone from 0 to 8, confidence
Out	Sensor samples	M2, UDP, to M6	Sensor data for the seismic graph, sent in small groups
Out	Status, delay reply	M2	Status on TCP once per second: frame rate, Wi-Fi signal, power, errors. Delay replies on UDP

4. Main components

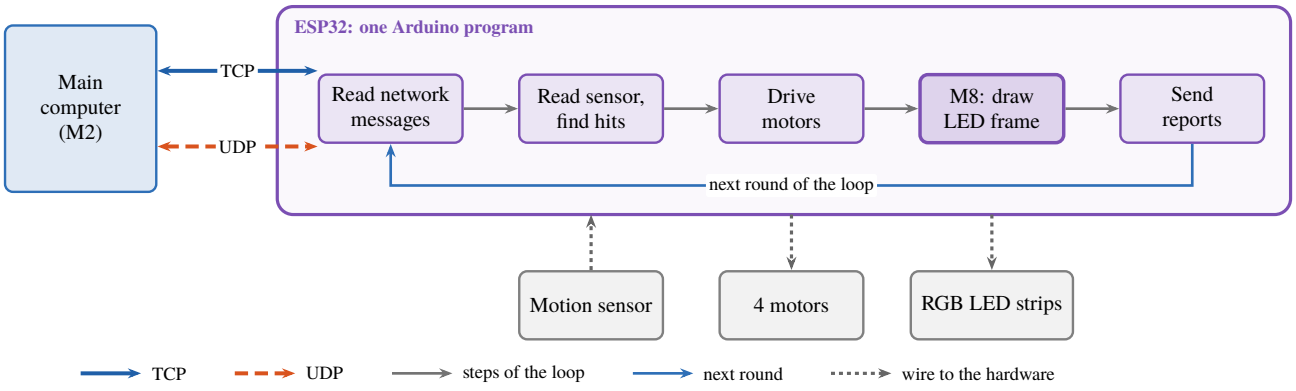
M7 has six parts. All of them run in the main loop of the Arduino program.

Component	Takes	Gives
Network link	Wi-Fi, server address	One TCP channel and one UDP stream, a new link after a loss
Message codec	Raw messages	Checked messages. Old, broken or wrong version messages are dropped
State hand over	Checked board state and config	The newest full state for M8
Hit detector	Sensor samples	Earthquake events with strength, zone and confidence
Motor controller	Vibrate commands	Motor power with a soft start and safety limits
Status and settings	Frame report, USB settings	Status reports, delay replies, settings saved in flash

5. How it works

One program with a main loop:

The ESP32 runs one Arduino program. M7 and M8 are two parts of it. The main loop repeats the same steps again and again: it reads the new network messages, reads the sensor, drives the motors, lets M8 draw one LED frame and sends the reports. M7 keeps the newest full board state in memory. At the start of each frame M8 reads this state, so it always draws the latest game.

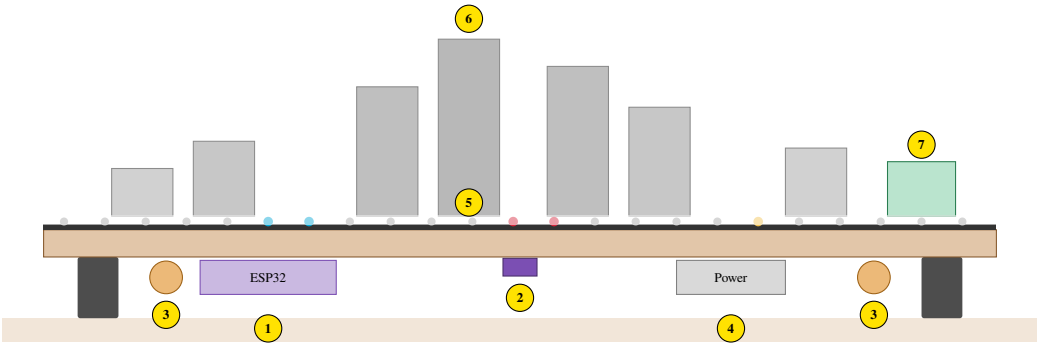


Two channels:

The board keeps one TCP channel and one UDP stream to the server. Messages that must arrive go on TCP: the config, the vibrate commands, the earthquake events and the status report. Each TCP message starts with its length, because TCP is only a stream of bytes. Data that comes again and again goes on UDP: the board state, the sensor samples and the delay probes. If one UDP packet is lost, the next one replaces it, so the LEDs never wait. Every message has an order number, so the board drops a UDP packet that comes late. Each start of the server makes a new session number. When the board sees a new session number, it starts its order numbers again, so it does not drop new messages as old ones.

What is inside the board:

The figure below is a side cut of the board. The motion sensor is fixed at the centre, under the plywood, so it feels a hit from every side. The four vibration motors are under the four corners. The ESP32 and a power supply for the LEDs and the motors sit under the board too.



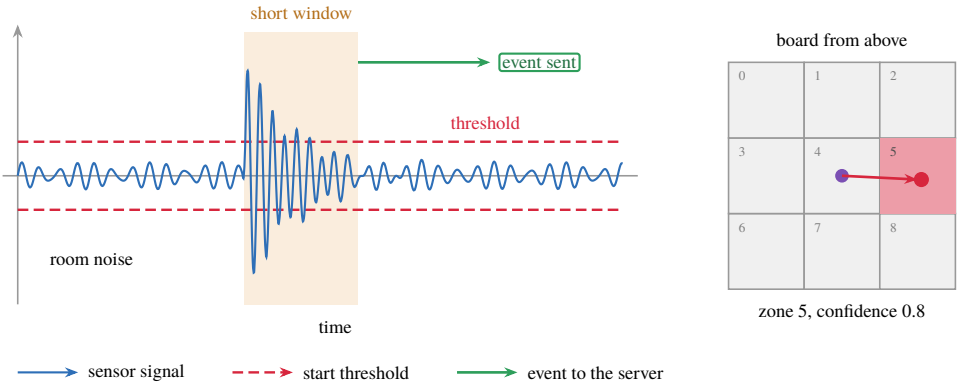
1 ESP32
2 Motion sensor at the centre
3 Vibration motors at the corners
4 Power supply
5 RGB LED strips on the roads
6 3D printed buildings
7 Team store with LEDs

Finding a hit:

The sensor measures the motion all the time. First a filter takes away gravity and slow tilting, so only the shaking is left. The detector also learns the normal noise of the room and puts the start threshold well above it. So the threshold goes up on a noisy table, and normal play does not start an earthquake. When the signal passes the threshold, a hit starts, and the detector takes a short window from this moment. The stronger the shaking in this window, the higher the strength. Like a real earthquake scale, the strength grows slowly for very strong hits, so a light tap and a hard hit both get a useful value. The scale is set once in calibration with a standard hit, and the result is kept between 0 and 10.

Finding the zone:

The accelerometer shows the direction of the first push of the board, and the gyroscope shows how the board tilts. Together they show the side of the hit, and the size of the push shows how far the hit is from the centre. From this the detector picks one of 9 zones in a 3 × 3 grid and gives a confidence value. After a hit the detector waits a short time, so one hit makes only one event.

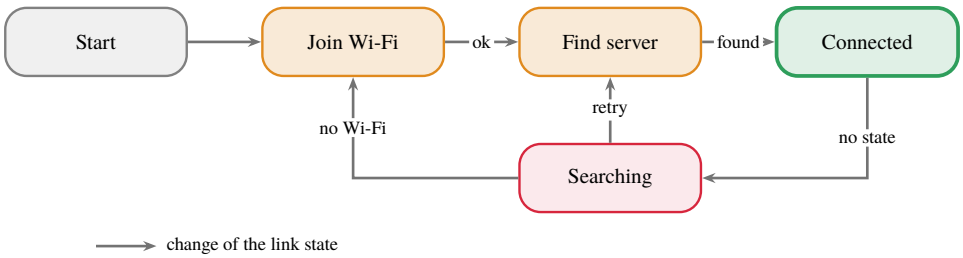


Motors:

The four motors are under the corners, at zones 0, 2, 6 and 8. When the admin presses the earthquake button, M1 sends a vibrate command on TCP. The command says which motors to use, the strength and the time. When the admin chooses one zone, M1 uses the motors nearest to it. For the whole board it uses all four. M7 starts the motors slowly, so the power does not jump. While the motors run, the detector stops learning the room noise, and the whole shaking gives only one event with "motors" as its source. So the earthquake in the game follows the real shaking of the table, and the motors do not start a second earthquake.

When the link is lost:

If no state comes for a short time, M7 stops the motors at once and tells M8 to show the "searching" animation. Then it tries to connect again: first to the server it finds on the local network, then to the backup address. The Wi-Fi name, the password, the backup address and the calibration values are stored in the ESP32's flash memory. We can change them over the USB cable, so we do not need to build the code again at a new place. M8 uses the same link state names.



6. Testing

M7 is tested alone with two PC programs. A server emulator takes the place of M2. It sends test board states, config and vibrate commands with delay and loss that we can change, and it shows the earthquake events and the seismic signal. A board emulator takes the place of the real board, so the server side can also be tested without it. We check the shared example messages, broken packets, a long run, heavy loss, a router restart and a server restart. For the sensor we hit each of the 9 zones many times and check that hits are found, that the zone is right or a neighbour zone, and that the motors do not start a false earthquake.